

CLOUD COMPUTING: NETWORKING

Module 4: Networking in Cloud

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-III



UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Outline

- 1 Network Fundamentals
- 2 Subnetting & CIDR
- 3 Virtual Networks in Cloud
- 4 Load Balancing & CDN



Outline

- 1 Network Fundamentals
- 2 Subnetting & CIDR
- 3 Virtual Networks in Cloud
- 4 Load Balancing & CDN



How Do Two Computers Communicate?

The Stack

Communication requires layers: **Physical** → **Network** → **Transport** →

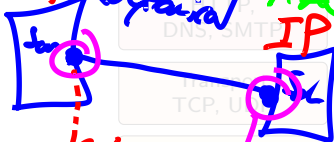
Application

MAC — IP/Port

Message
Payload
(JSON)

Proff. 1

MAC
IP
static
dynamic



socket
Port
Code

MAC (# network card)

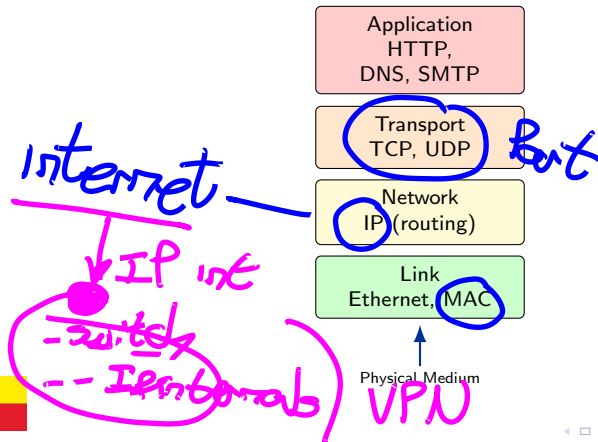
Proff. 2



How Do Two Computers Communicate?

The Stack

Communication requires layers: **Physical** → **Network** → **Transport** → **Application**.



Identifies a **device** on the network

IPv6: 2001:db8::1

Physical hardware identifier.

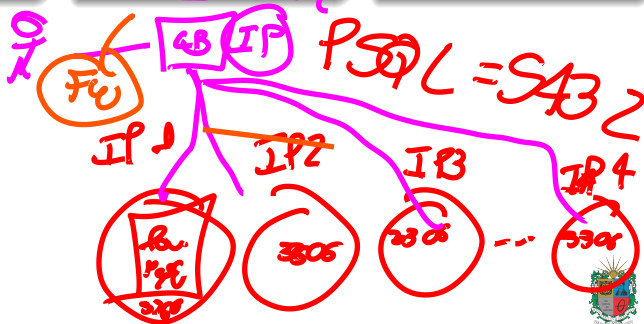
(used on local network only)

Endpoint for a service on a device.

443 = HTTPS

3306 = MySQL

Inside the Network



TCP vs UDP

TCP (Transmission Control Protocol)

- **Reliable:** guarantees delivery
- **Ordered:** packets arrive in sequence
- **Connection-based:** establish before sending
- Slower due to overhead
- Use: HTTP, SSH, Email

UDP (User Datagram Protocol)

- **Fast:** minimal overhead
- **Unreliable:** packets may be lost
- **Connectionless:** send without setup
- Lower latency
- Use: DNS, Video, Online gaming

X netw

Choose TCP for accuracy, UDP for speed.



TCP vs UDP

TCP (Transmission Control Protocol)

- **Reliable**: guarantees delivery
- **Ordered**: packets arrive in sequence
- **Connection-based**: establish before sending
- Slower due to overhead
- Use: HTTP, SSH, Email

UDP (User Datagram Protocol)

- **Fast**: minimal overhead
- **Unreliable**: packets may be lost
- **Connectionless**: send without setup
- Lower latency
- Use: DNS, Video, Online gaming

Choose TCP for accuracy, UDP for speed.



TCP vs UDP

TCP (Transmission Control Protocol)

- **Reliable**: guarantees delivery
- **Ordered**: packets arrive in sequence
- **Connection-based**: establish before sending
- Slower due to overhead
- Use: HTTP, SSH, Email

UDP (User Datagram Protocol)

- **Fast**: minimal overhead
- **Unreliable**: packets may be lost
- **Connectionless**: send without setup
- Lower latency
- Use: DNS, Video, Online gaming

Choose TCP for accuracy. UDP for speed.



Outline

- 1 Network Fundamentals
- 2 Subnetting & CIDR**
- 3 Virtual Networks in Cloud
- 4 Load Balancing & CDN



Why Divide Networks?

The Problem

A flat network of 1,000 computers broadcasts to all 1,000 devices. This creates **broadcast storms** and **poor performance**.

Solution: Subnetting

- Divide into smaller networks (*subnets*)
- Each subnet has its own broadcast domain
- Routers connect subnets
- Better security and performance



Why Divide Networks?

The Problem

A flat network of 1,000 computers broadcasts to all 1,000 devices. This creates **broadcast storms** and **poor performance**.

Solution: Subnetting

- Divide into smaller networks (*subnets*)
- Each subnet has its own broadcast domain
- Routers connect subnets
- Better security and performance



Why Divide Networks?

The Problem

A flat network of 1,000 computers broadcasts to all 1,000 devices. This creates **broadcast storms** and **poor performance**.

Solution: Subnetting

- Divide into smaller networks (*subnets*)
- Each subnet has its own **broadcast domain**
- Routers connect subnets
- Better security and performance



Why Divide Networks?

The Problem

A flat network of 1,000 computers broadcasts to all 1,000 devices. This creates **broadcast storms** and **poor performance**.

Solution: Subnetting

- Divide into smaller networks (*subnets*)
- Each subnet has its own **broadcast domain**
- Routers connect subnets
- Better security and performance



Why Divide Networks?

The Problem

A flat network of 1,000 computers broadcasts to all 1,000 devices. This creates **broadcast storms** and **poor performance**.

Solution: Subnetting

- Divide into smaller networks (*subnets*)
- Each subnet has its own **broadcast domain**
- Routers connect subnets
- Better **security** and **performance**



CIDR Notation

Classless Inter-Domain Routing

CIDR describes a network with *IP + prefix length*.

Example: 192.168.1.0/24 means first 24 *bits* are network, last 8 are host.

CIDR	Subnet Mask	Usable IPs
/32	255.255.255.255	1 (host)
/24	255.255.255.0	254
/16	255.255.0.0	65,534
/8	255.0.0.0	16M+

Common Cloud Networks

Private ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16



CIDR Notation

Classless Inter-Domain Routing

CIDR describes a network with *IP + prefix length*.

Example: 192.168.1.0/24 means first 24 *bits* are network, last 8 are host.

CIDR	Subnet Mask	Usable IPs
/32	255.255.255.255	1 (host)
/24	255.255.255.0	254
/16	255.255.0.0	65,534
/8	255.0.0.0	16M+

Common Cloud Networks

Private ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16



Outline

- 1 Network Fundamentals
- 2 Subnetting & CIDR
- 3 Virtual Networks in Cloud**
- 4 Load Balancing & CDN



VPC: Virtual Private Cloud

Definition

A **isolated network** within a *cloud provider*. You define subnets, routing, security rules—similar to owning a *private datacenter*.

Typical VPC Components:

- Subnets: public (internet-facing) or private
- Internet Gateway: connects VPC to public internet
- NAT Gateway: allows private subnet to initiate outbound traffic
- Route Tables: define how packets flow between subnets
- Security Groups: firewall rules for resources
- Network ACLs: network-level firewall



VPC: Virtual Private Cloud

Definition

A **isolated network** within a *cloud provider*. You define subnets, routing, security rules—similar to owning a *private datacenter*.

Typical VPC Components:

- **Subnets**: public (internet-facing) or private
- Internet Gateway: connects VPC to public internet
- NAT Gateway: allows private subnet to initiate outbound traffic
- Route Tables: define how packets flow between subnets
- Security Groups: firewall rules per resource
- Network ACLs: subnet-level firewall



VPC: Virtual Private Cloud

Definition

A **isolated network** within a *cloud provider*. You define subnets, routing, security rules—similar to owning a *private datacenter*.

Typical VPC Components:

- **Subnets**: public (internet-facing) or private
- **Internet Gateway**: connects VPC to public internet
- NAT Gateway: allows private subnet to initiate outbound traffic
- Route Tables: define how packets flow between subnets
- Security Groups: firewall rules per resource
- Network ACLs: subnet-level firewall



VPC: Virtual Private Cloud

Definition

A **isolated network** within a *cloud provider*. You define subnets, routing, security rules—similar to owning a *private datacenter*.

Typical VPC Components:

- **Subnets**: public (internet-facing) or private
- **Internet Gateway**: connects VPC to public internet
- **NAT Gateway**: allows private subnet to initiate outbound traffic
- Route Tables: define how packets flow between subnets
- Security Groups: firewall rules per resource
- Network ACLs: subnet-level firewall



VPC: Virtual Private Cloud

Definition

A **isolated network** within a *cloud provider*. You define subnets, routing, security rules—similar to owning a *private datacenter*.

Typical VPC Components:

- **Subnets**: public (internet-facing) or private
- **Internet Gateway**: connects VPC to public internet
- **NAT Gateway**: allows private subnet to initiate outbound traffic
- **Route Tables**: define how packets flow between subnets
- Security Groups: firewall rules per resource
- Network ACLs: subnet-level firewall



VPC: Virtual Private Cloud

Definition

A **isolated network** within a *cloud provider*. You define subnets, routing, security rules—similar to owning a *private datacenter*.

Typical VPC Components:

- **Subnets**: public (internet-facing) or private
- **Internet Gateway**: connects VPC to public internet
- **NAT Gateway**: allows private subnet to initiate outbound traffic
- **Route Tables**: define how packets flow between subnets
- **Security Groups**: firewall rules per resource
- **Network ACLs**: subnet-level firewall



VPC: Virtual Private Cloud

Definition

A **isolated network** within a *cloud provider*. You define subnets, routing, security rules—similar to owning a *private datacenter*.

Typical VPC Components:

- **Subnets**: public (internet-facing) or private
- **Internet Gateway**: connects VPC to public internet
- **NAT Gateway**: allows private subnet to initiate outbound traffic
- **Route Tables**: define how packets flow between subnets
- **Security Groups**: firewall rules per resource
- **Network ACLs**: subnet-level firewall



DNS: *Domain Name System*

The Problem Solved

Humans remember domain names (google.com), not IP addresses (142.250.80.46). **DNS** translates names to *IPs*.

DNS Hierarchy:

- Root nameserver: knows TLD servers
- TLD server: knows authoritative server for domain
- Authoritative server: holds actual host records (A, CNAME, etc.)
- Resolver (your laptop) DNS (caches results)

Example Record

A record: example.com → 192.0.2.1



DNS: *Domain Name System*

The Problem Solved

Humans remember domain names (google.com), not IP addresses (142.250.80.46). **DNS** translates names to *IPs*.

DNS Hierarchy:

- 1 **Root nameserver**: knows TLD servers
- 2 TLD server: knows authoritative server for domain
- 3 Authoritative server: holds actual DNS records (*A*, *CNAME*, *MX*)
- 4 Resolver: your ISP's DNS (caches results)

Example Record

A record: example.com → 192.0.2.1



DNS: *Domain Name System*

The Problem Solved

Humans remember domain names (google.com), not IP addresses (142.250.80.46). **DNS** translates names to *IPs*.

DNS Hierarchy:

- 1 **Root nameserver**: knows TLD servers
- 2 **TLD server**: knows authoritative server for domain
- 3 **Authoritative server**: holds actual DNS records (*A*, *CNAME*, *MX*)
- 4 **Resolver**: your ISP's DNS (caches results)

Example Record

A record: example.com → 192.0.2.1



DNS: *Domain Name System*

The Problem Solved

Humans remember domain names (google.com), not IP addresses (142.250.80.46). **DNS** translates names to *IPs*.

DNS Hierarchy:

- 1 **Root nameserver**: knows TLD servers
- 2 **TLD server**: knows authoritative server for domain
- 3 **Authoritative server**: holds actual DNS records (*A*, *CNAME*, *MX*)
- 4 **Resolver**: your ISP's DNS (caches results)

Example Record

A record: example.com → 192.0.2.1



DNS: *Domain Name System*

The Problem Solved

Humans remember domain names (google.com), not IP addresses (142.250.80.46). **DNS** translates names to *IPs*.

DNS Hierarchy:

- 1 **Root nameserver**: knows TLD servers
- 2 **TLD server**: knows authoritative server for domain
- 3 **Authoritative server**: holds actual DNS records (*A*, *CNAME*, *MX*)
- 4 **Resolver**: your ISP's DNS (caches results)

Example Record

A record: example.com → 192.0.2.1



DNS: *Domain Name System*

The Problem Solved

Humans remember domain names (google.com), not IP addresses (142.250.80.46). **DNS** translates names to *IPs*.

DNS Hierarchy:

- 1 **Root nameserver**: knows TLD servers
- 2 **TLD server**: knows authoritative server for domain
- 3 **Authoritative server**: holds actual DNS records (A, **CNAME**, **MX**)
- 4 **Resolver**: your **ISP's** **DNS** (caches results)

Example Record

A record: **example.com** → 192.0.2.1



Outline

- 1 Network Fundamentals
- 2 Subnetting & CIDR
- 3 Virtual Networks in Cloud
- 4 Load Balancing & CDN**



Load Balancing

The Problem

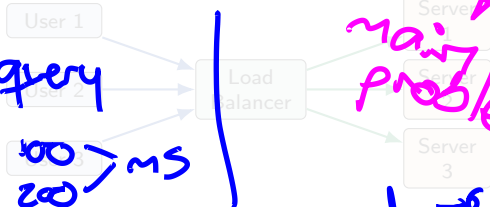
One server can only handle 1000 requests/second. Millions of users?
Distribute traffic across multiple servers.

1 request → 1 ms

DB → query

1000 > ms

Round Robin



main problem

$1 \times 10^6 = 1 \text{ qs}$

Strategies:

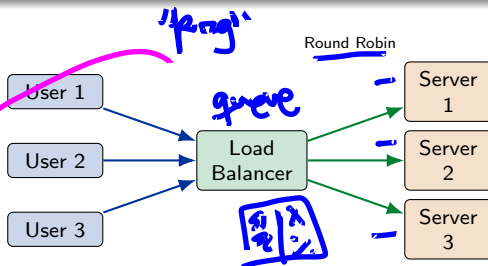
- Round robin, weighted routing, least connections
- health checks: stop sending traffic to failed servers



Load Balancing

The Problem

One server can only handle 1000 requests/second. Millions of users?
Distribute traffic across multiple servers.



Strategies:

- Round robin, weighted routing, least connections
- Health checks: stop sending traffic to failed servers

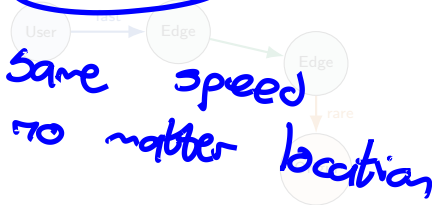


CDN: Content Delivery Network

The Problem

A user in Tokyo requests data from a US server = high latency.

Solution: cache content at edge locations globally.



Benefits:

- Reduced latency for users worldwide
- Reduced bandwidth costs
- DDoS protection at the edge

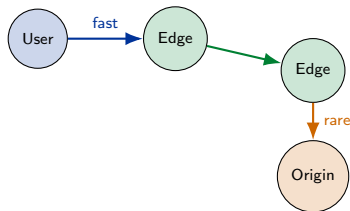


CDN: Content Delivery Network

The Problem

A user in *Tokyo* requests data from a *US* server = **high latency**.

Solution: **cache content** at edge locations globally.



Benefits:

- Reduced latency for users worldwide
- Reduced bandwidth costs
- DDoS protection at the edge

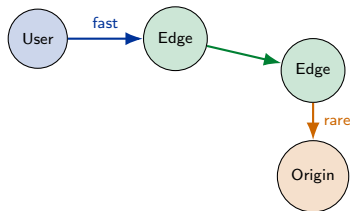


CDN: Content Delivery Network

The Problem

A user in *Tokyo* requests data from a *US* server = **high latency**.

Solution: **cache content** at edge locations globally.



Benefits:

- Reduced **latency** for users worldwide
- Reduced **bandwidth** costs

 **DDoS protection** at the edge



Outline

- 1 Network Fundamentals
- 2 Subnetting & CIDR
- 3 Virtual Networks in Cloud
- 4 Load Balancing & CDN



Recap: Module 4

- **TCP/UDP**: choose based on reliability vs. speed needs
- **CIDR**: $/24 = 254$ hosts, $/16 = 65k$ hosts, etc.
- **VPC**: isolated network with subnets, gateways, security
- **DNS**: translate domain names to IPs (distributed system)
- **Load Balancer**: distribute traffic across servers
- **CDN**: cache content at global edge locations for speed



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/cloud-computing>

